Programação Avançada

Unidade 2: Estruturas de Dados

Aula 06: Lista Encadeada

Prof. Eduardo Corrêa Gonçalves

27/04/2026

Sumário

Introdução

O que são Lista Encadeadas?

Criação

Propriedades e Operações

Principais Variações

Listas Duplamente Encadeadas

Listas Circulares

Link para Notebook

Introdução (1/3)

- **Motivação**
  - No Python, a ED **lista** é similar ao *array* de outras linguagens
  - A lista é uma estrutura de **acesso aleatório**.
    - Isso significa que o **acesso por posição** é feito em tempo **O(1)**.

[0]	[1]	[2]	[3]
John	Yoko	Julian	Sean

  - **Inserir** um elemento no **fim** (*append*) também tem complexidade amortizada* **O(1)**.

* Consiste na complexidade média do algoritmo considerando um número infinito de casos. Na aula 01, slides 44 a 47, vimos que o append é quase sempre O(1), excetuando a situação em que a capacidade da lista interna da estrutura está esgotada.

Introdução (2/3)

- **Motivação**
  - Por outro lado, a ED lista possui **três desvantagens**:

    1. Inserir um item **no início** leva **O(n)**, onde *n* é o número de elementos.

    2. **Buscar** por um **valor** é **O(n)**.

    3. Ela **exige** a alocação de **posições contíguas de memória** para que seja estruturada.
       - É isso que viabiliza a busca por posição em O(1).

[0]	[1]	[2]	[3]
John	Yoko	Julian	Sean

Introdução (3/3)

- **Motivação**
  - A **Lista Encadeada** (*linked list*) resolve os problemas (1) e (3)

    1. Inserir um item **no início** leva **O(n)**, onde *n* é o número de elementos.

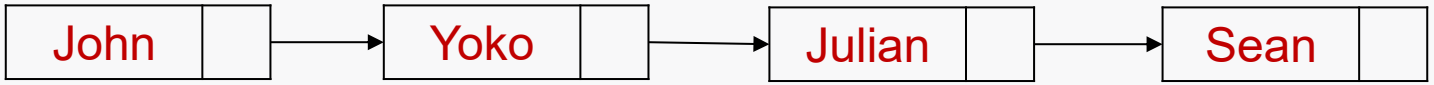
    2. **Buscar** por um **valor** é **O(n)**.

    3. Ela **exige** a alocação de **posições contíguas de memória** para que seja estruturada.

  - Porém, com uma desvantagem:
    - Ao resolver (3), sacrifica-se o acesso por posição em O(1) !!!

O que são Listas Encadeadas? (1/3)

- **Lista Encadeada**
  - Coleção ordenada de *n* elementos.

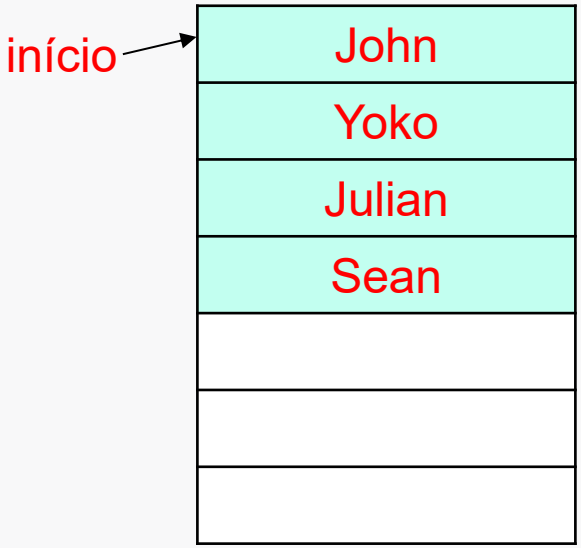


  - Cada elemento é um **nó**.
  - Os nós formam uma **cadeia**.
    - Isto é: cada nó possui um **link** com uma referência que aponta para o nó subsequente.
  - Características:
    - Permite elementos **duplicados**.
    - **Mutável** – pode ser alterada.
    - **Iterável** – capaz de retornar seus elementos um por vez em um laço.
    - **Sequência** – elementos possuem ordem determinada.

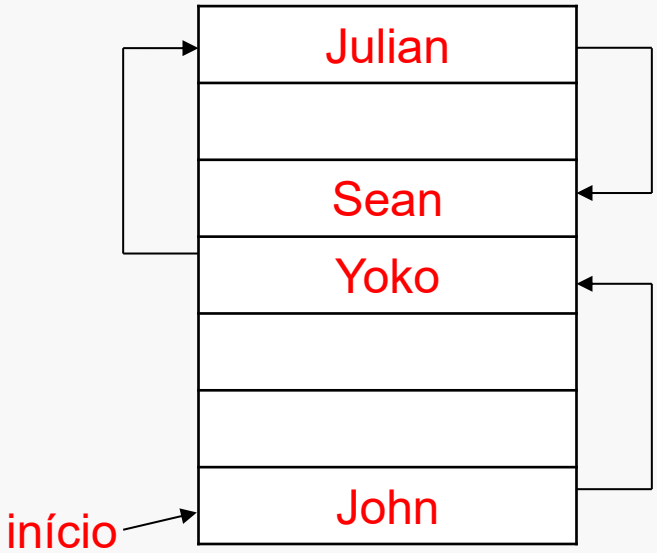
O que são Listas Encadeadas? (2/3)

- **Qual a Principal Diferença entre a Lista/Array e a Lista Encadeada?**
  - Resposta: a organização em memória.

- **Listas e Arrays** utilizam um bloco contíguo de memória para estruturar a sequência de dados.
  - Se não existir um bloco contíguo com tamanho suficiente para os elementos, não dá para criar ou estender a lista/array



- A **lista encadeada** armazena referências para o próximo elemento como parte de cada elemento.
  - Assim, **não** é necessário alocar um bloco contíguo.



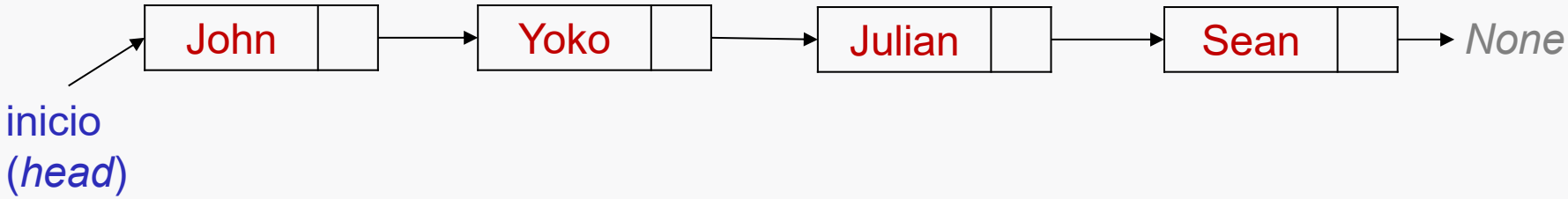
O que são Listas Encadeadas? (3/3)

- **Estrutura de um Nó da Lista Encadeada**
  - Em uma lista encadeada, cada nó possui dois diferentes campos:
    - **Dados** (***data*** ou ***info***): contém o valor do nó.
    - **Próximo** (***next*** ou ***link***): contém a referência para o próximo nó da lista.

Imagem retirada de: Pregueiro (2020)

Criação de Listas Encadeadas (1/6)

- **Implementando a nossa própria lista encadeada**
  - Nesta aula, implementaremos a nossa própria lista encadeada utilizando POO.



Criação de Listas Encadeadas (2/6)

- É preciso definir duas classes:

  - **No**: representação de um nó da lista encadeada (info + próximo)

```
class No:

    def __init__(self, dados):
        self.info = dados
        self.prox = None
```

  - **ListaEncadeada**: p/ representar a lista propriamente dita (cadeia de nós)

```
class ListaEncadeada:

    def __init__(self):
        #ponteiro para o início
        self.IL = None

        #num. de elementos
        self.N = 0
```

Criação de Listas Encadeadas (3/6)

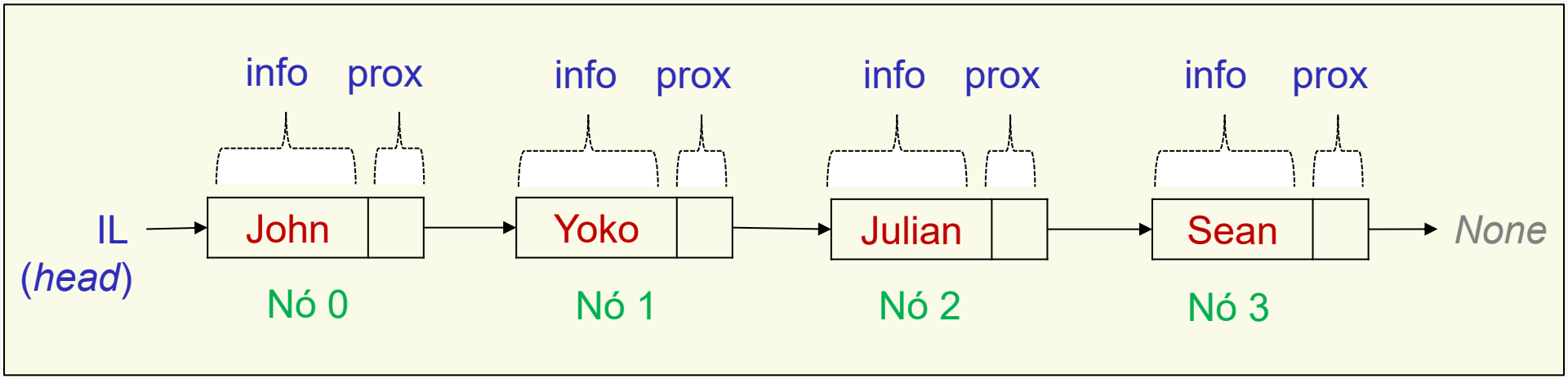
- É preciso definir duas classes :

  - **No**: representação de um nó da lista encadeada (info + próximo)

```
class No:
    def __init__(self, dados):
        self.info = dados
        self.prox = None
```

  - **ListaEncadeada**: p/ representar a lista propriamente dita (cadeia de nós)

```
class ListaEncadeada:
    def __init__(self):
        #ponteiro para o início
        self.IL = None
        #num. de elementos
        self.N = 0
```



Criação de Listas Encadeadas (4/6)

- Com o método mágico __repr__, é possível definir a representação string da lista.

```
class ListaEncadeada:
    …

 # representação string da lista
   def __repr__(self):
      no_atu = self.IL
      aux = []
      while no_atu is not None:
         aux.append(str(no_atu.info))
         no_atu = no_atu.prox

      aux.append("<FL>")
      return " -> ".join(aux)
```

O método __repr__ é quase igual ao __str__. Para saber a exata diferença entre ambos, consulte: https://www.python-course.eu/python3_object_oriented_programming.php

Criação de Listas Encadeadas (5/6)

- Testando...

```
>>> lst = ListaEncadeada()
>>> no_0 = No("John")
>>> lst.IL = no_0
>>> lst.N = 1

>>> print(lst)
John → <None>

>>> no_1 = No("Yoko")
>>> no_2 = No("Julian")
>>> no_3 = No("Sean")

>>> no_0.prox = no_1
>>> no_1.prox = no_2
>>> no_2.prox = no_3
>>> lst.N = 4

>>> print(lst)
John → Yoko → Julian → Sean → <None>
```

Criação de Listas Encadeadas (6/6)

- Outro método útil é o que testa se uma lista está vazia.

- Ele será utilizado em dentro de vários outros métodos.

```
class ListaEncadeada:
    …

# lista vazia?
def vazia(self):
    return self.IL is None
```

Operações (1/11)

- A lista encadeada será implementada com os métodos apresentados na tabela abaixo (*todos definidos na classe ListaEncadeada*).
  - A tabela também fornece uma comparação entre as complexidades na Lista Encadeada (coluna 2) e na lista tradicional (coluna 3).
  - Relembrando que:
    - Principal **vantagem** da Lista Encadeada → não requer espaço contíguo de memória.
    - Principal **desvantagem** da Lista Encadeada → busca por posição é O(n)

Operação	Lista Encadeada	Lista do Python
Criação	O(1)	O(1)
Inserir / Remover no Início	O(1)	**O(n)**
Inserir / Remover no Fim	**O(n)***	O(1)
Inserir / Remover no "meio"	O(n)	O(n)
Buscar elemento pela posição	**O(n)**	O(1)
Pesquisar se um valor existe na lista	O(n)	O(n)

* Essa desvantagem é fácil de contornar, bastando adicionar um ponteiro para o fim da lista

Operações (2/11)

- **Inserir no início (algoritmo)**
  - Lista original "lst":

    inicio → John → Yoko → Julian → Sean → *None*

  - Criamos um novo nó: Paul → *None*
  - Fazemos o "próximo" do novo nó apontar para o início da lista

    inicio → John → Yoko → Julian → Sean → *None*

    ↑ **Paul**

  - Por fim, fazemos o início da lista apontar para novo nó

    John → Yoko → Julian → Sean → *None*

    inicio → Paul

Operações (3/11)

- **Inserir no início** (*é sempre O(1) e não O(1) amortizado !!!*)

```
def inserir_inicio(self, novo_no):
    novo_no.prox = self.IL
    self.IL = novo_no
    self.N += 1
    return True
```

 # Ex.: insere "Paul" no início da lista encadeada

John → Yoko → Julian → Sean → <None>

```
lst.inserir_inicio(No("Paul"))
print(lst)
```

Paul → John → Yoko → Julian → Sean → <None>

Operações (4/11)

- **Inserir no fim (algoritmo)**
  - Criamos um novo nó: Ringo → *None*
  - Percorremos a lista original "lst" até o último nó

    John → Yoko → Julian → **Sean** → *None*

    inicio
  - E fazemos próximo do último nó apontar para novo nó

    John → Yoko → Julian → Sean

    inicio

    Ringo → *None*

Operações (5/11)

- **Inserir no fim** (O(n), exceto se tivermos ponteiro FL para o fim da lista)

```
# insere elemento no fim da lista
def inserir_fim(self, novo_no):
    if (self.vazia()): #lista ainda vazia
        self.IL = novo_no

    else: # lista tem ao menos um elemento
            # preciso navegar até o último nó
        no_atu = self.IL
        while (no_atu.prox is not None): no_atu = no_atu.prox
        no_atu.prox = novo_no

    self.N += 1
    return True
```

```
# Ex.: insere "Ringo" no fim da lista John → Yoko → Julian → Sean → <None>
lst.inserir_fim(No("Ringo"))
print(lst)     #John → Yoko → Julian → Sean → Ringo → <None>
```

Operações (6/11)

- **Inserir em (exemplo inserir na <u>posição i = 2</u>).** ***Obs:*** *o 1º elemento está na posição i=0.*
  - Criamos um novo nó: George → *None*
  - Percorremos a lista original "lst" até o elemento *i-1*

    John → **<u>Yoko</u>** → Julian → Sean → *None*

    inicio
  - Fazemos próximo do novo nó apontar para o nó *i*

    John → Yoko → Julian → Sean → *None*

    inicio George
  - Fazemos próximo do nó *i-1* apontar para o novo nó

    John → Yoko Julian → Sean → *None*

    inicio George

Operações (7/11)

- **Inserir na posição i**

```
# insere na posição k da lista
def inserir_em(self, k, novo_no):
    if (k > self.N) or (k < 0): return False

    if (self.vazia()): # lista ainda vazia – basta fazer o início apontar para o novo nó
        self.IL = novo_no
    elif(k == 0): # se a posição é 0,  basta inserir no início
        novo_no.prox = self.IL
        self.IL = novo_no
    else: # se lista possui ao menos um elemento e posição não é 0, uso o algoritmo
        no_ant = self.IL;
        cont = 0
        while (cont != k-1): # vai até a posição
            no_ant = no_ant.prox
            cont += 1

    novo_no.prox = no_ant.prox
    no_ant.prox = novo_no

    self.N += 1
    return True
```

Operações (7/11)

- **Inserir na posição i**

 # Ex.: insere "George" na posição 2
John → Yoko → Julian → Sean → <None>

```
lst.inserir_em(2, No("George"))
print(lst)
```

John -> Yoko -> George -> Julian -> Sean -> <None>

Operações (8/11)

- **Recuperar elemento da posição i** (*é O(n), a grande desvantagem*)

```
# recuperar elemento da posição k
def recuperar(self, k):
    if (self.vazia()) or (k >= self.N) or (k < 0): return False

    no_atu = self.IL
    cont = 0
    while (cont != k):
        no_atu = no_atu.prox
        cont += 1

    return no_atu.info
```

```
# Ex.: recupera elemento da posição 1
John → Yoko → Julian → Sean → <None>

print(lst.recuperar(1))
> Yoko
```

Operações (9/11)

- **Remover do início (algoritmo** – que é O(1)**)**
  - Dada a lista original "lst"

    John → Yoko → Julian → Sean → *None*

    inicio

  - Fazemos início apontar para o próximo do Nó 0 e depois removemos o Nó 0

    ~~John~~ Yoko → Julian → Sean → *None*

    inicio

Operações (10/11)

- **Remover da posição (algoritmo – exemplo <u>posição</u> <u>i = 2</u>)** (complexidade O(n))
  - Dada a lista original "lst"

    John → Yoko → Julian → Sean → *None*

    inicio

  - Navegamos até a posição *i*

    John → Yoko → **<u>Julian</u>** → Sean → *None*

    inicio

    Nó anterior Nó i

  - Fazemos o nó anterior ao Nó *i* apontar para o próximo do Nó *i* e depois removemos o Nó *i*

    John → Yoko ~~Julian~~ Sean → *None*

    inicio

Operações (11/11)

- **Remover elemento da posição i**

```
#remove nodo da posiçao k
def remover(self, k):
    if (self.vazia()) or (k >= self.N) or (k < 0): return False

    if (k==0): #remoção do 1o nó - preciso mexer no início da lista

        if (self.N > 1):
            no_aux = self.IL
            self.IL = no_aux.prox
        else:
            self.IL = None

        no_aux = None  # "mata" o nó0 – na verdade, nem é necessário no Python
                       # devido ao garbage collector.
                       # Em outras linguagens é necessário
```

CONTINUA ---->

Operações (11/11)

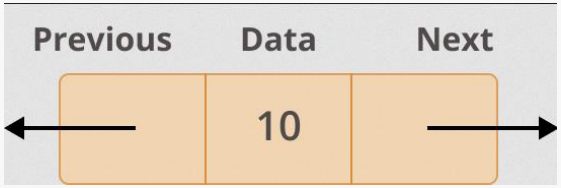
- **Remover elemento da posição i (continuação)**

```
else: #remoção de qualquer outro nó que não seja o 1º
    no_ant = None
    no_atu = self.IL
    cont = 0
    while (cont != k):
        no_ant = no_atu
        no_atu = no_atu.prox
        cont += 1

    no_ant.prox = no_atu.prox
    no_atu = None #"mata" o nó k – nem é algo realmente necessário no Python...

self.N -= 1
return True
```

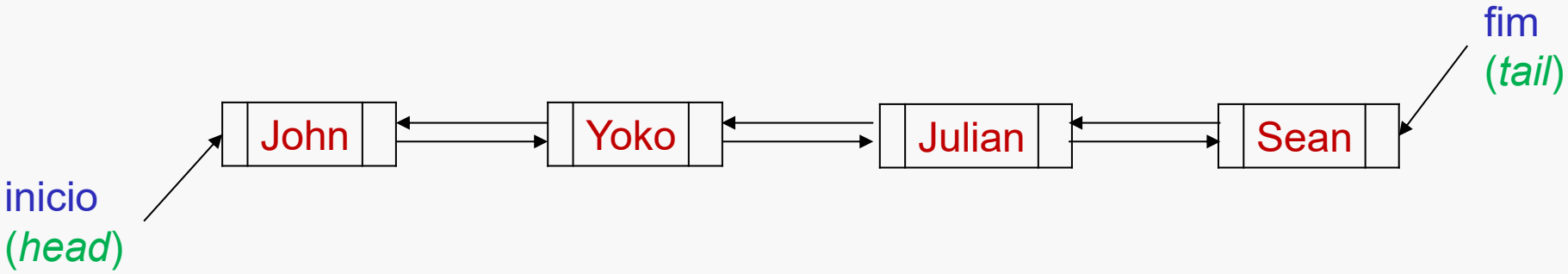
```
# Ex.:
John → Yoko → Julian → Sean → <None>

lst.remover(2)
John → Yoko → Sean → <None>
```

Outros Tipos de Lista Encadeada (1/3)

- **Lista Duplamente Encadeada**
  - Possui duas referências: uma para o elemento anterior e outra para o elemento seguinte.

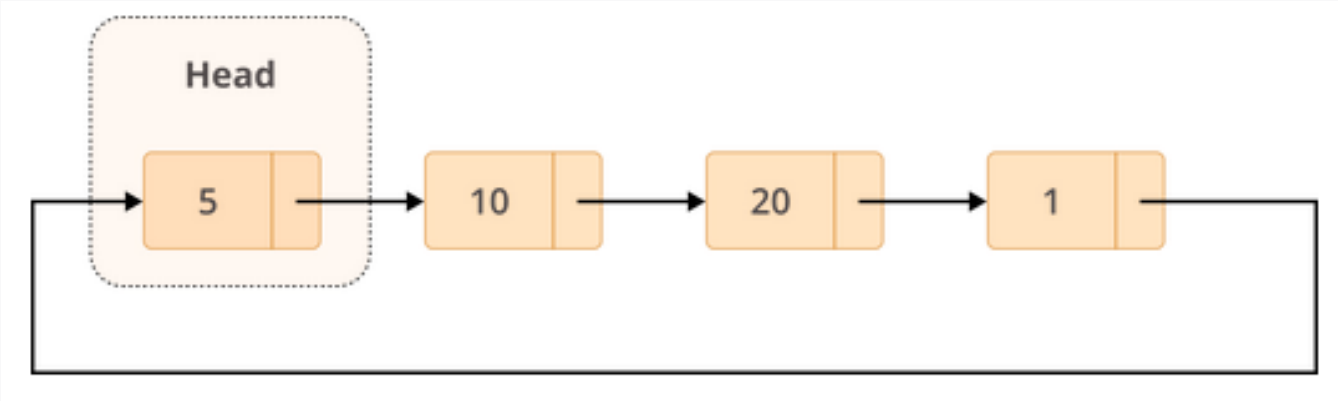


  - **Vantagens**:
    - Usa um ponteiro no último elemento (fim), possibilitando a inserção no final em O(1). Isso dá para fazer na outra também.
    - Torna possível percorrer a lista em 2 sentidos.



Outros Tipos de Lista Encadeada (2/3)

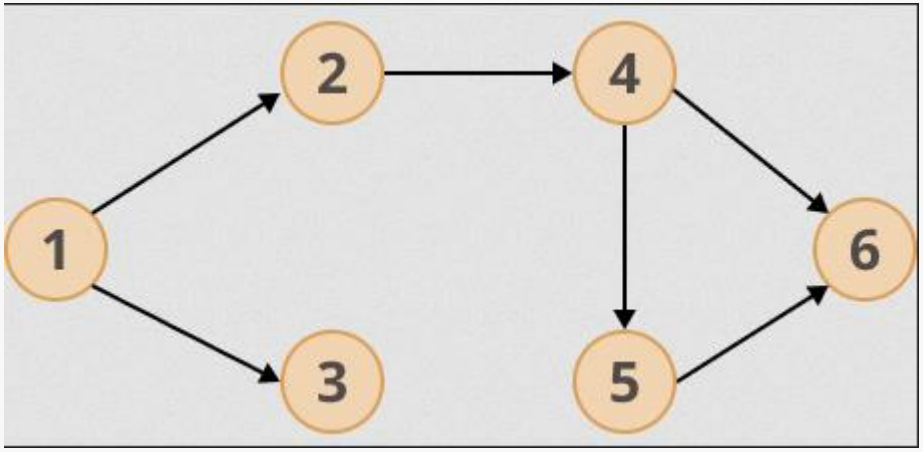
- **Lista Circular**
  - Tipo de lista em que o último elemento possui um ponteiro de volta para o primeiro.



  - Boa opção para problemas onde os diferentes elementos serão utilizados em turnos, durante várias iterações.
    - **Exemplo**: em um jogo de cartas, pode ser usada para implementar a vez de cada participante realizar a sua jogada (após o último jogar, volta novamente ao primeiro)

Outros Tipos de Lista Encadeada (3/3)

- **Importância dos Diferentes Tipos de Listas**
  - Acabamos de apresentar 2 diferentes variações de implementação das listas.
  - Mas é possível criar muitas outras.
  - Dependendo da adaptação, poderemos usar listas encadeadas como base para representar pilhas, filas, árvores, grafos e outras estruturas.



Link para Notebook no Colab

- **Lista Encadeada**
  - https://colab.research.google.com/drive/1PgHq14CRU_DzgBQ_LfXDnYJX8OOUbizs?usp=sharing

Resumo

- **Lista Encadeada**
  - **Pontos fortes**:
    - Não exige espaço contíguo de memória para alocação
    - A inserção e remoção no início tem custo O(1).
    - É flexível:
      - Há diferentes tipos de lista encadeada.
      - Há diferentes formas de implementar.
      - Listas encadeadas são utilizadas como base para a implementação de estruturas mais complexas, como árvores.
  - **Pontos fracos**:
    - Recuperar elemento pela posição é O(n). → *pior desvantagem !!!*
    - Inserir no fim é O(n)
      - ... Exceto se guardamos ponteiro para o fim da lista também
      - ... Ou se usarmos a lista duplamente encadeada

Exercício

(1) Crie o seguinte método para a classe ListaEncadeada:

- **busca_valor(v)**: recebe como entrada um valor v (ex: “Yoko”) e retorna como **saída** a **posição do primeiro nó** que armazene esse valor (ex: 1).
- Caso o valor não exista na lista ou a lista esteja vazia, o método deve retornar *None*.
- A implementação será **O(n)**, já que sempre é preciso sair do primeiro nó.

(2) (ESPECIAL) Crie um método **inserir_fim(v)** (ou **append**...) com complexidade **O(1)**.

- Para tal, você precisará acrescentar um ponteiro “FL” na classe ListaEncadeada sempre apontando para o último elemento da lista.
- Além disso, será preciso alterar o código dos métodos de inserção e remoção de elementos apresentados nessa aula.

Referências

- Edelweiss, N. e Galante, R. (2009). “Estruturas de Dados”. Bookman

- Pregueiro, P. (2020). Linked Lists in Python: an Introduction. https://realpython.com/linked-lists-python/

- Python Docs (2020). Collections – Container datatypes https://docs.python.org/3/library/collections.html